

Universidad de Guayaquil

Ingeniería en Telemática

Programa de Control y Administración de Producción de Llenado de Productos Secos orientado a RTOS

Materia: Sistemas Operativos

Docente: Ing. Freddy Pincay, Mgs.

Período 2026-2027 · Ciclo I · Parcial 2

Integrante:

Adrián Ariel Damián Vallejo

6 de julio de 2026

Índice

1. Levantamiento de requisitos y análisis	3
1.1 El problema y el alcance	3
1.2 Requisitos funcionales	3
1.3 Requisitos no funcionales	3
1.4 Riesgos que identificamos	4
2. Diseño del sistema	5
2.1 Base de datos	5
2.2 Arquitectura	5
2.3 La máquina de estados del llenado	6
2.4 Comunicación MQTT	6
2.5 Diseño de la interfaz	6
3. Desarrollo del programa	8
3.1 Codificación	8
3.2 Manual técnico (resumen)	8
3.3 Manual de usuario (resumen)	8
4. Emulación y pruebas	11
4.1 El entorno de emulación	11
4.2 Pruebas automatizadas	11
4.3 Prueba de extremo a extremo	11
4.4 Métricas de tiempo real	11
5. Mantenimiento y mejora continua	13
6. Bibliografía (APA 7)	14

1. Levantamiento de requisitos y análisis

1.1 El problema y el alcance

La idea del proyecto es controlar una máquina que llena fundas de productos secos, en nuestro caso maní y pasas, en presentaciones de 25 y 50 gramos, y que se producen por lotes de 50 o 100 unidades según el pedido. Cuando empezamos a analizar el problema nos dimos cuenta de que en realidad son dos sistemas en uno: por un lado está la máquina, que tiene que pesar y cerrar la válvula en el momento justo (ahí es donde entra el tiempo real), y por otro lado está la parte administrativa de siempre: órdenes de producción, inventario, usuarios y reportes.

Por eso decidimos partir el sistema en dos programas que se comunican por MQTT: un firmware que corre sobre FreeRTOS en un ESP32 (emulado en Wokwi porque no tenemos la máquina física) y un backend web en Python que lleva la administración y guarda todo en una base de datos PostgreSQL.

1.2 Requisitos funcionales

Código	Requisito	Dónde se cumple
RF-01	Iniciar y detener ciclos de llenado	Botones físicos y web (cmd MQTT)
RF-02	Seleccionar producto y presentación (25/50 g)	Formulario de órdenes
RF-03	Abrir/cerrar la válvula automáticamente	Tarea de control (FSM)
RF-04	Validar peso objetivo $\pm$ tolerancia (0.6 g)	Estado VALIDANDO de la FSM
RF-05	Registrar cada funda llenada (trazabilidad)	Tabla historico_produccion
RF-06	Gestionar órdenes con estados	pendiente → en_proceso → completada
RF-07	Controlar inventario de materias primas	Descuento automático por unidad OK
RF-08	Generar reportes y exportarlos	Módulo de reportes + CSV
RF-09	Usuarios con roles	operador / supervisor / gerente
RF-10	Monitoreo en tiempo real para el operador	Panel web (Socket.IO) y LCD

1.3 Requisitos no funcionales

- Rendimiento: el sistema debe aguantar hasta 50 órdenes de producción por día. Con lo que medimos (una unidad cada ~2 segundos) sobra capacidad de procesamiento.
- Determinismo: el lazo báscula → control → válvula tiene que responder en tiempo acotado. El periodo de control es de 25 ms y el peor ciclo que medimos fue de 3.9 ms, o sea que siempre

llega a tiempo.

- Seguridad: acceso con usuario y clave (hash), y cada rol ve solo lo que le corresponde.
- Usabilidad: el operador solo necesita dos botones en la máquina y un panel web que se entiende de un vistazo.
- Disponibilidad: si el broker MQTT se cae, la máquina sigue produciendo sola; la telemetría se descarta y no bloquea el control.

1.4 Riesgos que identificamos

- No tenemos hardware real: lo resolvimos emulando todo en Wokwi, que trae el sensor HX711 con celda de carga incluida.
- El broker MQTT es público (broker.hivemq.com), así que puede tener latencia o caerse. Como la lógica de control vive en el ESP32, esto no afecta el llenado, solo el monitoreo.
- El plan gratuito de Wokwi CI corta cada simulación a los 5 minutos; para demos largas usamos la extensión de VS Code.
- La calibración de la báscula se puede desajustar; dejamos el factor de escala como constante configurable y documentamos cómo recalibrar.

2. Diseño del sistema

2.1 Base de datos

Usamos las seis tablas que pide el proyecto, normalizadas y con sus claves foráneas y restricciones CHECK (por ejemplo, la presentación solo puede ser 25 o 50 y el stock no puede quedar negativo). El script completo está en db/esquema.sql del repositorio.

Tabla	Qué guarda	Campos principales
productos	Catálogo	nombre, presentacion_gr (25/50)
ordenes_produccion	Pedidos	producto, tam_lote, cantidad, estado, operador
lotes	Lotes producidos	numero_lote, fechas de producción y caducidad (+180 días), producidas, rechazadas
inventario_materias_primas	Stock	materia_prima, cantidad_disponible, unidad
historico_produccion	Una fila por funda	lote, peso_real, ok, fecha_hora, operador
usuarios	Acceso	usuario, clave_hash, rol

2.2 Arquitectura

La arquitectura es hexagonal (puertos y adaptadores) pero sin carpetas anidadas: el dominio (las reglas del negocio, como validar el peso o calcular el consumo de materia prima) no sabe nada de MQTT, de la base de datos ni de FreeRTOS. Todo lo externo son adaptadores que se conectan por interfaces bien definidas.

Algo que nos pareció interesante explicar: FreeRTOS no aparece como una "capa" del diagrama, porque no lo es. Es el motor que ejecuta cada adaptador y el núcleo como tareas concurrentes con prioridades. Y hay una inversión curiosa: en la arquitectura la HMI "manda" (es quien dispara los casos de uso), pero en el firmware corre con la prioridad más baja, porque el humano tolera latencia y el lazo de control no.

Tarea FreeRTOS	Rol	Prioridad	Disparo
tarea_control	Núcleo: FSM y validación	5 (máx.)	ciclo de 25 ms
tarea_bascula	Adaptador sensor HX711	4	periódica, 50 ms
tarea_valvula	Adaptador actuador (relé)	4	por evento (cola)
tarea_alarmas	Vigilancia de tolerancia y fallos	3	event group
tarea_mqtt	Telemetría hacia el backend	2	cola + 500 ms

Tarea FreeRTOS	Rol	Prioridad	Disparo
tarea_hmi	LCD y métricas por serial	1 (mín.)	periódica, 200 ms

La sincronización entre tareas se hace con los mecanismos del RTOS: una cola de longitud 1 para el peso (siempre la muestra más fresca), colas para comandos de válvula y telemetría, un mutex para el estado compartido de la orden, un event group para las alarmas y semáforos binarios que se liberan desde las interrupciones de los botones. No hay variables compartidas sin proteger.

2.3 La máquina de estados del llenado

El ciclo de cada funda pasa por: ESPERA → DOSIFICANDO (válvula abierta) → ASENTANDO (esperamos 400 ms a que el peso se estabilice) → VALIDANDO (¿está dentro de objetivo ± 0.6 g?) → DESCARGA (cambio de funda) y vuelta a empezar. Un detalle que aprendimos probando: hay que cerrar la válvula ANTES de llegar al objetivo (0.8 g antes), porque el producto que ya cayó de la válvula pero todavía no llegó a la funda sigue sumando peso. Si esperábamos al objetivo exacto, todas las fundas salían pasadas.

2.4 Comunicación MQTT

Tópico	Sentido	Contenido
envasadora/ENV01/peso	máquina → web	peso actual, 2 veces por segundo
envasadora/ENV01/estado	máquina → web	estado FSM, lote, contadores (retained)
envasadora/ENV01/unidad	máquina → web	cada funda: peso final, aceptada o no
envasadora/ENV01/alarma	máquina → web	fuera de tolerancia, fallo de sensor
envasadora/ENV01/cmd	web → máquina	iniciar orden (producto, lote) o parar

Elegimos un broker público porque Render (donde se despliega el backend) no acepta conexiones TCP entrantes que no sean HTTP, así que no podíamos hospedar el broker ahí. El backend se conecta como un cliente más.

2.5 Diseño de la interfaz

Para la HMI de la máquina usamos un LCD 16x2 que muestra el estado, el avance del lote y el peso en vivo, más dos botones (INICIO y PARO) y dos LED (unidad aceptada / alarma). Para la parte web hicimos un panel oscuro tipo industrial con el peso en grande, el estado de la máquina, los contadores del lote y el inventario, todo actualizándose solo por WebSocket, sin recargar la página.

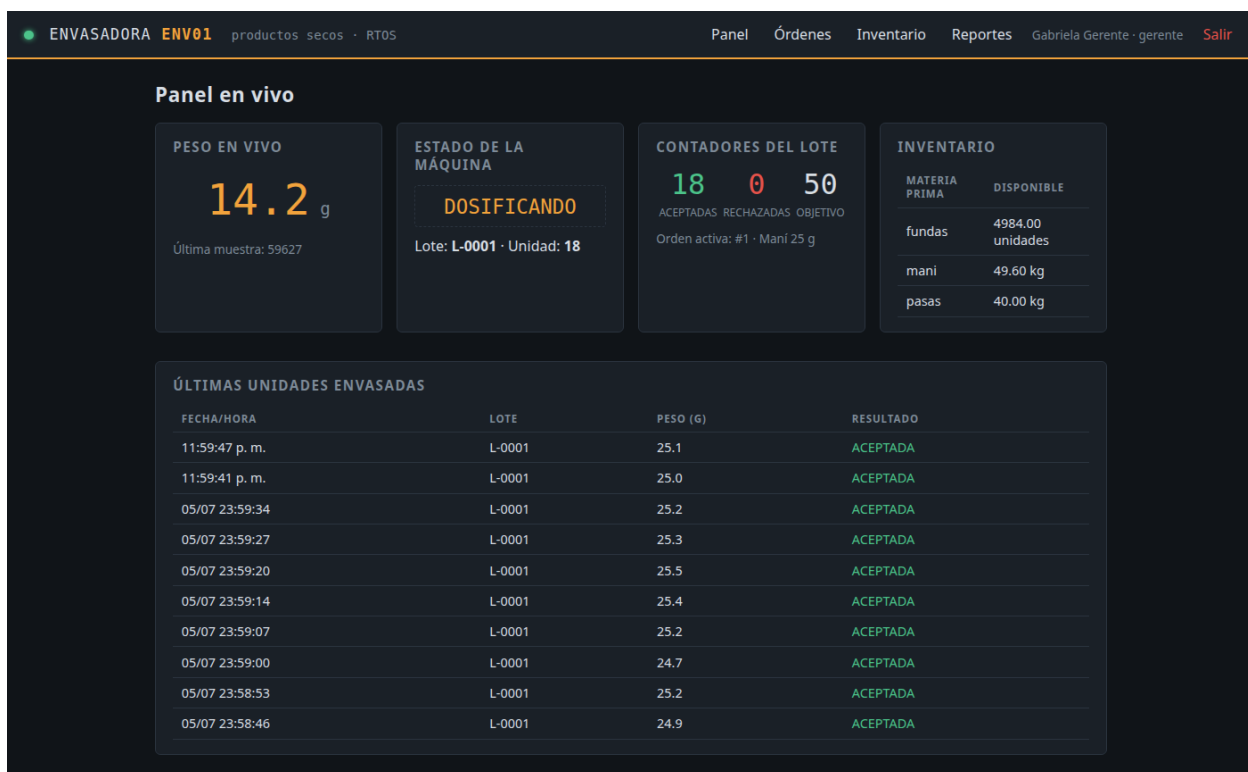


Figura 1. Panel en vivo durante la producción del lote L-0001.

3. Desarrollo del programa

3.1 Codificación

El firmware está en C++ con el framework de Arduino para ESP32 (unas 400 líneas en `firmware/src/main.cpp`), usando directamente la API de FreeRTOS que viene integrada: `xTaskCreatePinnedToCore`, colas, mutex, event groups y semáforos desde ISR. Las tareas de control van fijadas al núcleo 1 del ESP32 y la de red al núcleo 0, para que el WiFi no le robe tiempo al lazo de control.

El backend está en Python con Flask 3 y SQLAlchemy 2, separado en dominio (reglas puras), adaptadores (MQTT entrante y repositorio de base de datos) y vistas (los módulos web). La telemetría entra por Flask-MQTT y se reenvía al navegador con Socket.IO. Antes de escribir código investigamos proyectos maduros para no empezar de cero: la librería HX711 de Bogdan Necula, PubSubClient para MQTT, y varios proyectos de básculas IoT de donde sacamos el patrón de calibración y el esquema de tópicos (el detalle está en `docs/HALLAZGOS.md`).

No todo salió a la primera. Los tres problemas que más nos costaron: primero, las fundas salían todas bajas de peso hasta que modelamos el producto en vuelo; segundo, la alarma reportaba el peso leído tarde (cuando la funda ya se había descargado marcaba 0.08 g) y hubo que capturar el peso en el instante de la falla; y tercero, el firmware numeraba los lotes desde 1 en cada reinicio y se desincronizaba de la base de datos, así que ahora el backend manda el número de lote dentro del comando y la máquina lo respeta.

3.2 Manual técnico (resumen)

- Compilar el firmware: `cd firmware && pio run` (PlatformIO descarga el toolchain solo la primera vez).
- Simular: abrir la carpeta `firmware/` con la extensión Wokwi de VS Code, o por consola: `wokwi-cli . --scenario escenarios/llenado_basico.scenario.yaml`
- Backend local: `cd backend && pip install -r requirements.txt && python seeds.py && python app.py` → `http://localhost:5000`
- Base de datos: SQLite automática en local; en producción se usa PostgreSQL leyendo la variable `DATABASE_URL` (el esquema está en `db/esquema.sql`).
- Despliegue: `backend/render.yaml` define el servicio web en Render con gunicorn de UN solo worker (Flask-MQTT mantiene una única conexión al broker; el paralelismo va por hilos).
- Calibración de báscula: ajustar `FACTOR_ESCALA` en `main.cpp` con una masa conocida (`set_scale + tare` de la librería HX711).
- Parámetros de proceso: `TOLERANCIA_G` (0.6), `ANTICIPO_G` (0.8), `MS_ASENTAR` (400) y `MS_DESCARGA` (500) están como constantes al inicio del `main.cpp`.

3.3 Manual de usuario (resumen)

En la máquina: el botón verde INICIO arranca la orden (o reanuda después de un paro), el botón rojo PARO detiene el ciclo y cierra la válvula. El LCD muestra en qué estado está la máquina, cuántas unidades van del lote y el peso actual. El LED verde parpadea con cada funda aceptada y el rojo

avisa las alarmas.

En la web: se entra con usuario y clave. El operador ve el panel en vivo; el supervisor además crea órdenes (elige producto, presentación y tamaño de lote) y las inicia con un botón, que es lo que manda el comando a la máquina; el gerente ve también los reportes y puede descargarlos en CSV. El inventario se descuenta solo: no hay que registrar nada a mano.

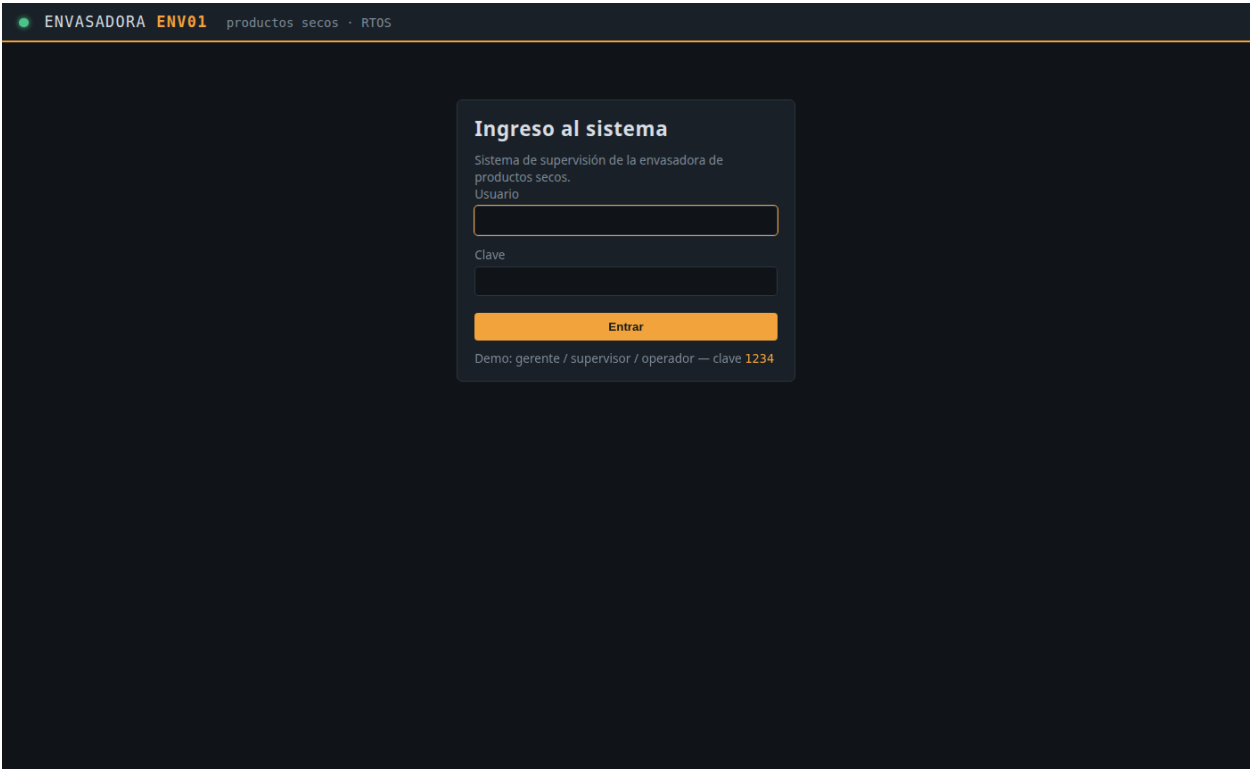


Figura 2. Acceso al sistema con roles.

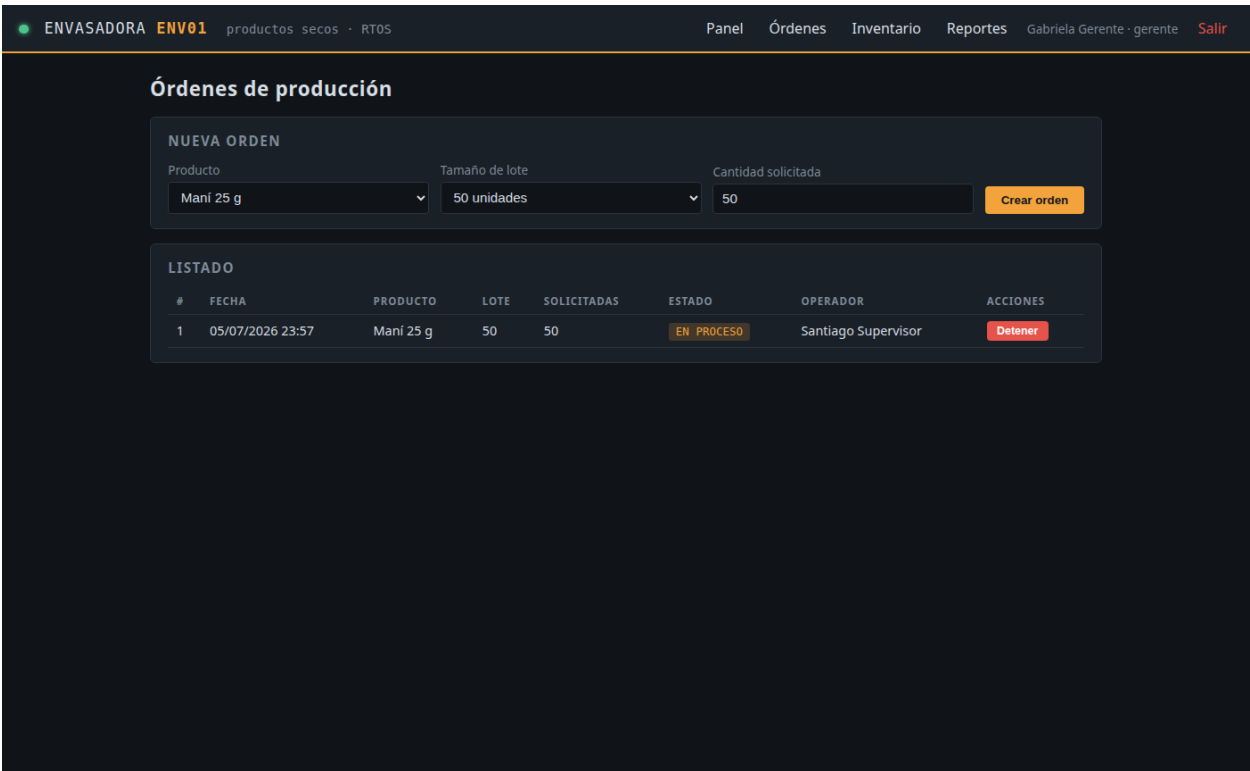


Figura 3. Gestión de órdenes de producción.

ENVASADORA ENV01 productos secos · RT05

PanelÓrdenesInventarioReportesGabriela Gerente · gerenteSalir

Inventario de materias primas

MATERIA PRIMA	DISPONIBLE	UNIDAD	AJUSTAR A
fundas	4982.00	unidades	4982,0 Guardar
mani	49.55	kg	49,5500000000000026 Guardar
pasas	40.00	kg	40,0 Guardar

El consumo automático se descuenta con cada unidad aceptada reportada por la envasadora (tópico **unidad**).

Figura 4. Inventario de materias primas.

4. Emulación y pruebas

4.1 El entorno de emulación

Toda la máquina corre emulada en Wokwi: el ESP32, el sensor de peso HX711 con su celda de carga de 5 kg, el relé que hace de válvula, el LCD y los botones. El circuito está descrito en `firmware/diagram.json`. Para poder demostrar el sistema sin estar moviendo el peso a mano, el firmware tiene un modo demo (MODO_DEMO) que simula la física del llenado: un flujo de unos 12 g/s con ruido, más el material en vuelo al cerrar la válvula. Apagando ese flag, la lectura vuelve a salir del HX711 real.



Figura 5. HMI física (LCD) capturada del framebuffer durante la emulación: dosificando la unidad 2 de 50, con 8.44 g en la báscula.

4.2 Pruebas automatizadas

Además de probar a mano, dejamos escenarios de prueba que corren con `wokwi-cli` en modo headless: el escenario arranca la simulación, pulsa el botón INICIO, y verifica por el puerto serial que la orden inicia, que se producen unidades y que las métricas de determinismo se publican. El resultado fue "Scenario completed successfully". También se prueba el caso de rechazo: cuando una funda sale fuera de tolerancia el sistema la cuenta aparte, enciende la alarma y publica el evento.

4.3 Prueba de extremo a extremo

La prueba más completa fue en vivo con todo conectado de verdad: desde la web creamos e iniciamos una orden; el backend publicó el comando en `broker.hivemq.com`; el ESP32 emulado lo recibió y empezó a producir; la telemetría volvió por MQTT y quedó en la base de datos. Los números cuadraron exactos:

Verificación	Resultado
Unidades registradas en histórico	13 unidades, todas aceptadas, peso promedio 25.14 g
Lote L-0001	13 producidas, 0 rechazadas
Inventario de maní	50 kg → 49.675 kg (13 × 25 g, exacto)
Inventario de fundas	5000 → 4987 (una por unidad)

4.4 Métricas de tiempo real

El firmware mide el peor tiempo de ejecución de cada ciclo y lo reporta cada 10 segundos. En todas las corridas el peor ciclo de la tarea de control fue de 1.7 a 3.9 milisegundos, contra un periodo de 25 ms: nunca se perdió un plazo. La memoria libre (heap) se mantuvo estable en ~222 KB durante toda la producción, señal de que no hay fugas.

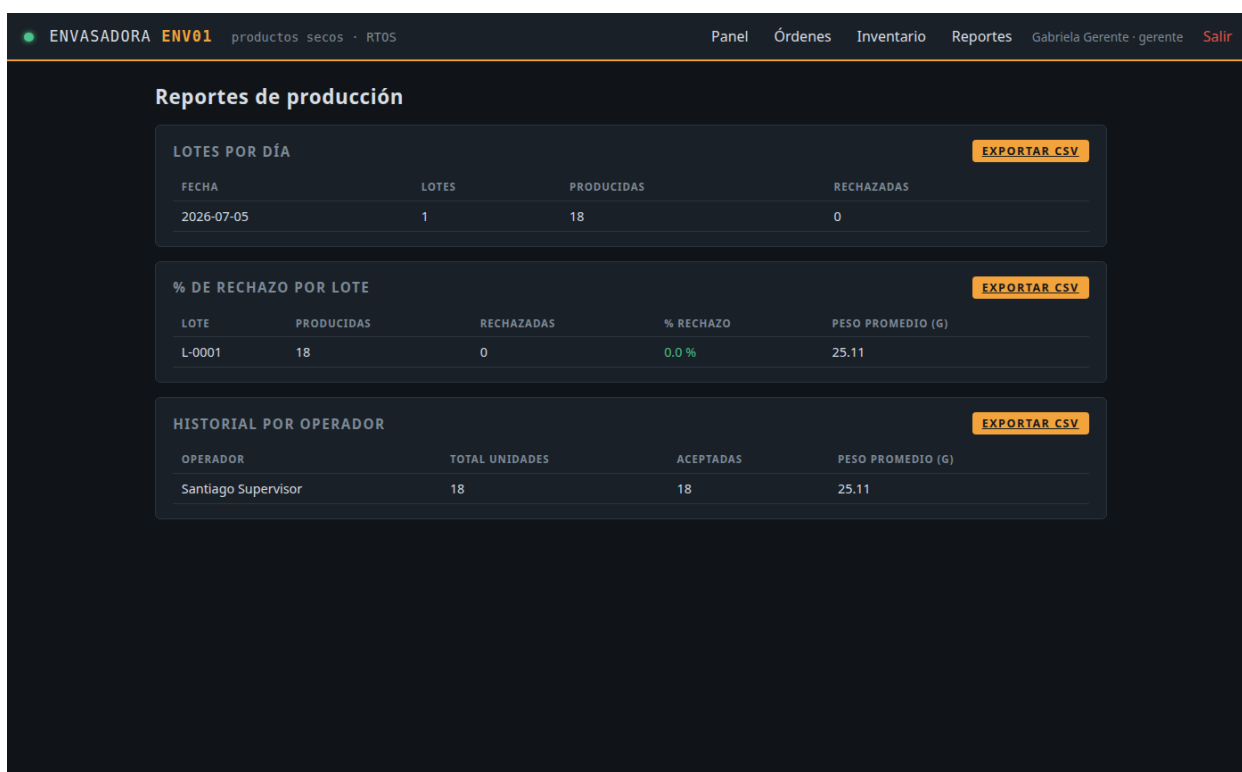


Figura 6. Reportes de producción con los datos reales de la emulación.

5. Mantenimiento y mejora continua

- Preventivo: recalibrar la báscula por turno con una masa patrón (procedimiento del manual técnico) y revisar las métricas de determinismo del serial, que delatan degradación antes de que falle.
- Correctivo: los errores quedan trazados en tres lugares (serial de la máquina, log del backend y tabla de históricos), así que se puede reconstruir qué pasó con cualquier funda.
- Evolutivo: quedan planteadas mejoras concretas: HMI SCADA con FUXA (ya evaluada en la investigación), autenticación en el broker MQTT, y soporte de más productos, que solo requiere insertar filas en el catálogo y su materia prima.
- Soporte: el README del repositorio tiene los comandos exactos para levantar cada parte, y los manuales están en este mismo documento.

6. Bibliografía (APA 7)

- Amazon Web Services. (2026). FreeRTOS documentation. <https://www.freertos.org/Documentation>
- Barry, R. (2016). Mastering the FreeRTOS real time kernel: A hands-on tutorial guide. Real Time Engineers Ltd.
- Buttazzo, G. (2011). Hard real-time computing systems: Predictable scheduling algorithms and applications (3.^a ed.). Springer.
- CodeMagic LTD. (2026). Wokwi documentation: HX711 load cell. <https://docs.wokwi.com/parts/wokwi-hx711>
- Espressif Systems. (2026). ESP32 Arduino core documentation. <https://docs.espressif.com/projects/arduino-esp32/>
- Grinberg, M. (2018). Flask web development: Developing web applications with Python (2.^a ed.). O'Reilly Media.
- HiveMQ GmbH. (2026). MQTT essentials. <https://www.hivemq.com/mqtt/>
- Necula, B. (2023). HX711: An Arduino library to interface the Avia Semiconductor HX711 [Software]. GitHub. <https://github.com/bogde/HX711>
- O'Leary, N. (2026). PubSubClient: A client library for MQTT messaging [Software]. GitHub. <https://github.com/knolleary/pubsubclient>
- OASIS. (2019). MQTT version 5.0 specification. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- Render Services, Inc. (2026). Render documentation: Web services. <https://render.com/docs/web-services>
- Tanenbaum, A. S., y Bos, H. (2015). Modern operating systems (4.^a ed.). Pearson.